

**LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 2**

**MODUL 14
SORTING**



Disusun oleh:

MANGGALA PATRA RADITYA

109082500179

S1IF-13-02

Asisten Praktikum

Muhammad Agha Zulfadhli

Renisa Assyifa Putri

**PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO**

2026

MODUL 14

1. Soal 1 Selection Sort

Source code

```
package main

import "fmt"

func selectionSort(arr []int) {
    n := len(arr)

    for i := 0; i < n-1; i++ {
        minIdx := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }

        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    var n int

    fmt.Scan(&n)
```

```
    for i := 0; i < n; i++ {  
        var m int  
        fmt.Scan(&m)  
  
        rumah := make([]int, m)  
  
        for j := 0; j < m; j++ {  
            fmt.Scan(&rumah[j])  
        }  
  
        selectionSort(rumah)  
  
        for j := 0; j < m; j++ {  
            if j > 0 {  
                fmt.Print(" ")  
            }  
            fmt.Print(rumah[j])  
        }  
        fmt.Println()  
    }  
}
```

Screenshoot program

The screenshot shows a Go IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'MODUL 14' with several files, including 'Soal1.go'. The code editor displays the following Go code for a Selection Sort algorithm:

```
1 package main
2
3 import "fmt"
4
5 func selectionSort(arr []int) {
6     n := len(arr)
7
8     for i := 0; i < n-1; i++ {
9         minIdx := i
10
11         for j := i + 1; j < n; j++ {
12             if arr[j] < arr[minIdx] {
13                 minIdx = j
14             }
15         }
16
17         arr[i], arr[minIdx] = arr[minIdx], arr[i]
18     }
19 }
```

The terminal shows the command to run the program and its output:

```
PS C:\Users\User\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14> go run "c:\Users\User\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14\Soal Selection Sort 1\Soal1.go"
3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1
1 2 7 9 13
15 27 39 75 133 189
1 4 9
```

Deskripsi program

Program ini digunakan untuk mengurutkan nomor rumah kerabat Hercules di setiap daerah secara menaik menggunakan algoritma **Selection Sort**. Program membaca jumlah daerah (n), kemudian untuk setiap daerah membaca jumlah rumah (m) beserta nomor rumahnya, lalu mencari elemen terkecil secara berulang dan menukarnya ke posisi yang tepat hingga seluruh data terurut. Setelah proses pengurutan selesai, program menampilkan nomor rumah yang telah tersusun dari yang terkecil hingga terbesar pada setiap daerah.

2. Soal 2 Selection Sort

Source code

```
package main

import "fmt"

func selectionSortAsc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func selectionSortDesc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        maxIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[maxIdx] {
```

```
        maxIdx = j

    }

}

arr[i], arr[maxIdx] = arr[maxIdx], arr[i]

}

}
```

```
func main() {

    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        var m int

        fmt.Scan(&m)

        var ganjil, genap []int

        for j := 0; j < m; j++ {

            var x int

            fmt.Scan(&x)

            if x%2 == 1 {

                ganjil = append(ganjil, x)

            } else {
```

```
        genap = append(genap, x)

    }

}

selectionSortAsc(ganjil)
selectionSortDesc(genap)

first := true

for _, v := range ganjil {
    if !first {
        fmt.Print(" ")
    }
    fmt.Print(v)
    first = false
}

for _, v := range genap {
    if !first {
        fmt.Print(" ")
    }
    fmt.Print(v)
    first = false
}
```

```

        fmt.Println()
    }
}

```

Screenshoot program

The screenshot shows a Go IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor displays a Go program implementing a Selection Sort algorithm. The terminal shows the output of running the program, which is a list of numbers: 3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1 1 7 9 13 2 15 27 39 75 133 189 1 9 4. The file explorer shows a project structure with a folder named 'MODUL 14' containing several files, including 'soal2.go'. The terminal shows the command 'go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14\Soal Selection Sort 2\soal2.go"' and the resulting output.

```

package main

import "fmt"

func selectionSortAsc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    // ... (code for reading input and calling selectionSortAsc)
}

```

```

PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14> go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14\Soal Selection Sort 2\soal2.go"
3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1
1 7 9 13 2
15 27 39 75 133 189
1 9 4
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14>

```

Deskripsi program

Program **kerabat dekat** digunakan untuk menampilkan nomor rumah kerabat Hercules dengan urutan semua bilangan **ganjil terlebih dahulu secara menaik**, kemudian diikuti semua bilangan **genap secara menurun**. Program membaca jumlah daerah dan nomor rumah pada setiap daerah, memisahkan bilangan ganjil dan genap ke dalam dua array berbeda, mengurutkan bilangan ganjil menggunakan **Selection Sort** secara ascending dan bilangan genap secara descending, lalu mencetak hasil gabungan sesuai format yang diminta pada setiap daerah.

3. Soal 3 Selection Sort

Source code

```
package main

import "fmt"

func insertionSort(arr []int) {
    for i := 1; i < len(arr); i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}

func main() {
    var x int
    data := []int{}

    for {
        fmt.Scan(&x)
```

```

        if x == -5313 {
            break
        }
        if x == 0 {
            if len(data) > 0 {
                insertionSort(data)

                n := len(data)
                var median int

                if n%2 == 1 {
                    median = data[n/2]
                } else {
                    median = (data[n/2-1] +
data[n/2]) / 2
                }

                fmt.Println(median)
            }
        } else {
            data = append(data, x)
        }
    }
}

```

Screenshoot program

The screenshot shows a Go IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'MODUL 14' with files 'Soal Insertion Sort 1', 'Soal Insertion Sort 2', 'Soal Selection Sort 1', 'Soal1.go', 'Soal Selection Sort 2', 'soal2.go', and 'Soal Selection Sort 3'. The code editor shows the following Go code:

```
1 package main
2
3 import "fmt"
4
5 func insertionSort(arr []int) {
6     for i := 1; i < len(arr); i++ {
7         key := arr[i]
8         j := i - 1
9
10        for j >= 0 && arr[j] > key {
11            arr[j+1] = arr[j]
12            j--
13        }
14        arr[j+1] = key
15    }
16 }
17
```

The terminal shows the command to run the program and its output:

```
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14> go run "c:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14\Soal Selection Sort 3\soal3.go"
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12
PS C:\Users\user\OneDrive\Dokumen\CODINGAN MANGGALA\Modul 14>
```

Deskripsi program

Program **median** digunakan untuk menghitung dan menampilkan nilai median dari seluruh data yang telah dibaca setiap kali ditemukan angka **0**. Data berupa bilangan bulat positif disimpan ke dalam array, kemudian saat angka 0 muncul, seluruh data yang telah tersimpan diurutkan menggunakan algoritma **Insertion Sort**. Setelah data terurut, median ditentukan berdasarkan jumlah data: jika jumlah data ganjil maka median adalah nilai tengah, sedangkan jika jumlah data genap maka median adalah rata-rata dua nilai tengah yang dibulatkan ke bawah. Program akan terus membaca data hingga menemukan penanda akhir **-5313**.

1. Soal 1 Insertion Sort

Source code

```
package main

import "fmt"

func insertionSort(arr []int) {
    for i := 1; i < len(arr); i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }
}

func main() {
    var x int
    data := []int{}

    for {
```

```
    fmt.Scan(&x)

    if x < 0 {
        break
    }

    data = append(data, x)
}

insertionSort(data)

for i := 0; i < len(data); i++ {
    if i > 0 {
        fmt.Print(" ")
    }

    fmt.Print(data[i])
}

fmt.Println()

tetap := true

for i := 1; i < len(data); i++ {
    if data[i]-data[i-1] != data[1]-data[0] {
        tetap = false
        break
    }
}
```

```

    }

}

if tetap {

    fmt.Println("Data berjarak tetap")

} else {

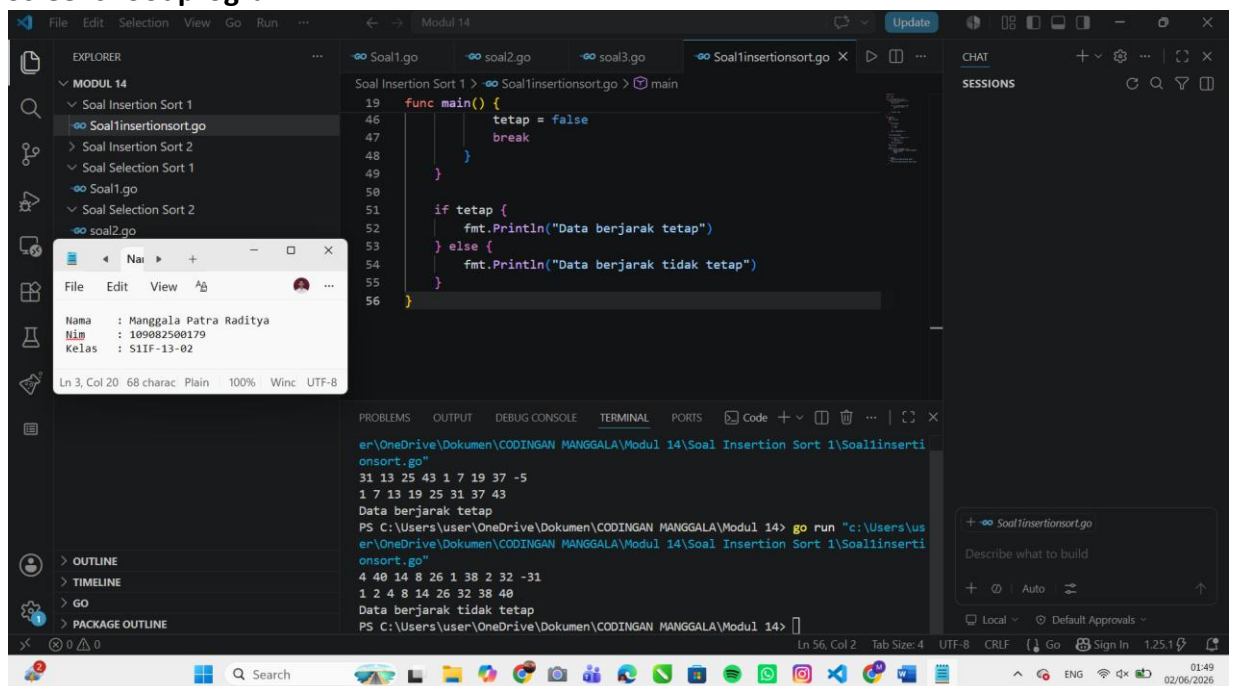
    fmt.Println("Data berjarak tidak tetap")

}

}

```

Screenshoot program



Deskripsi program

Program ini digunakan untuk membaca sekumpulan bilangan bulat non-negatif yang diakhiri oleh bilangan negatif. Seluruh bilangan non-negatif disimpan ke dalam array dan kemudian diurutkan secara menaik menggunakan algoritma **Insertion Sort**. Setelah data terurut, program memeriksa apakah selisih antara setiap dua elemen yang berurutan selalu sama. Jika semua selisih bernilai sama, program menampilkan pesan **"Data berjarak tetap"**; jika tidak, program menampilkan **"Data berjarak tidak tetap"**. Hasil keluaran terdiri dari data yang telah diurutkan dan status jarak antar datanya.

2. Soal 2 Insertion Sort

Source code

```
package main

import "fmt"

const NMAX = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku struct {
    data []Buku
}

func InputBuku(P *DaftarBuku, n int) {
    P.data = make([]Buku, n)

    for i := 0; i < n; i++ {
        fmt.Scan(
            &P.data[i].id,
            &P.data[i].judul,
            &P.data[i].penulis,
```



```

        &P.data[i].penerbit,
        &P.data[i].eksemplar,
        &P.data[i].tahun,
        &P.data[i].rating,
    )
}

}

func CetakTerfavorit(P DaftarBuku, n int) {
    if n > len(P.data) {
        n = len(P.data)
    }

    for i := 0; i < n; i++ {
        fmt.Println(
            P.data[i].id,
            P.data[i].judul,
            P.data[i].penulis,
            P.data[i].penerbit,
            P.data[i].eksemplar,
            P.data[i].tahun,
            P.data[i].rating,
        )
    }
}

```

```

}

func UrutBuku(P *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := P.data[i]
        j := i - 1

        for j >= 0 && P.data[j].rating < temp.rating {
            P.data[j+1] = P.data[j]
            j--
        }

        P.data[j+1] = temp
    }
}

func CetakTerbaru(P DaftarBuku, n int) {
    if n > len(P.data) {
        n = len(P.data)
    }

    for i := 0; i < n; i++ {
        fmt.Println(
            P.data[i].id,

```

```

        P.data[i].judul,
        P.data[i].penulis,
        P.data[i].penerbit,
        P.data[i].eksemplar,
        P.data[i].tahun,
        P.data[i].rating,
    )
}
}

func CariBuku(P DaftarBuku, n int, rating int) {
    left := 0
    right := n - 1
    idx := -1

    for left <= right {
        mid := (left + right) / 2

        if P.data[mid].rating == rating {
            idx = mid
            break
        } else if P.data[mid].rating < rating {
            right = mid - 1
        } else {

```

```

        left = mid + 1

    }

}

if idx == -1 {

    fmt.Println("Buku tidak ditemukan")

} else {

    fmt.Println(

        P.data[idx].id,

        P.data[idx].judul,

        P.data[idx].penulis,

        P.data[idx].penerbit,

        P.data[idx].eksemplar,

        P.data[idx].tahun,

        P.data[idx].rating,

    )

}

}

func main() {

    var n int

    fmt.Scan(&n)

    var pustaka DaftarBuku

```

```

InputBuku(&pustaka, n)

UrutBuku(&pustaka, n)

CetakTerfavorit(pustaka, 5)

CetakTerbaru(pustaka, 5)

var ratingCari int

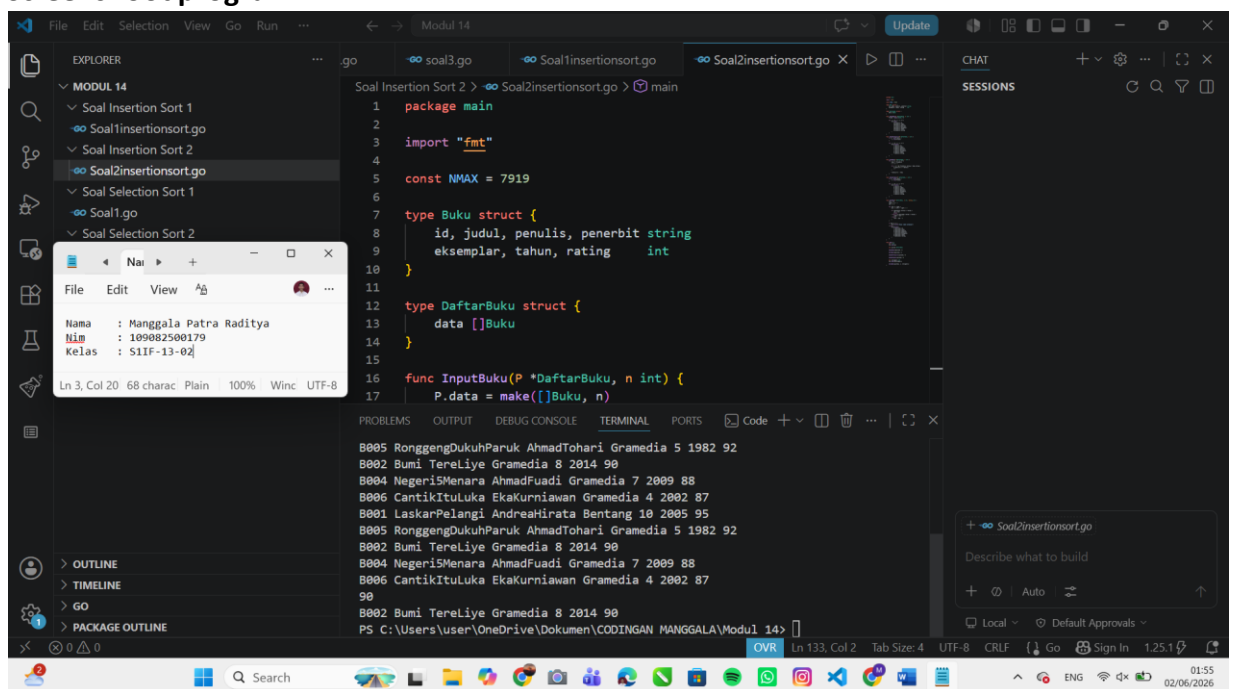
fmt.Scan(&ratingCari)

CariBuku(pustaka, n, ratingCari)

}

```

Screenshoot program



Deskripsi program

Program ini digunakan untuk mengelola data buku perpustakaan yang terdiri dari ID, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Data buku dibaca dan disimpan ke dalam array bertipe `DaftarBuku`, kemudian diurutkan berdasarkan rating tertinggi menggunakan algoritma **Insertion Sort**. Setelah diurutkan, program menampilkan lima buku dengan rating tertinggi, menampilkan data buku tertentu, serta melakukan pencarian buku berdasarkan rating menggunakan **Binary Search** agar proses pencarian lebih cepat dan efisien.

pseudocode subprogram

```
program Perpustakaan

const NMAX = 7919

type Buku :
    < id, judul, penulis, penerbit : string,
        eksemplar, tahun, rating : integer >

type DaftarBuku : array[1..NMAX] of Buku

procedure InputBuku(in/out pustaka : DaftarBuku, n :
integer)

procedure UrutBuku(in/out pustaka : DaftarBuku, n :
integer)

procedure CetakTerfavorit(in pustaka : DaftarBuku, n :
integer)

procedure CariBuku(in pustaka : DaftarBuku, n : integer,
rating : integer)

kamus
```

```

    pustaka : DaftarBuku
    n, ratingCari : integer

algoritma
    input(n)

    InputBuku(pustaka, n)

    UrutBuku(pustaka, n)

    CetakTerfavorit(pustaka, 5)

    input(ratingCari)

    CariBuku(pustaka, n, ratingCari)

endprogram

procedure InputBuku(in/out pustaka : DaftarBuku, n :
integer)
kamus
    i : integer
algoritma
    for i <- 1 to n do

```

```

        input(pustaka[i].id,
              pustaka[i].judul,
              pustaka[i].penulis,
              pustaka[i].penerbit,
              pustaka[i].eksemplar,
              pustaka[i].tahun,
              pustaka[i].rating)

    endfor
endprocedure

procedure UrutBuku(in/out pustaka : DaftarBuku, n :
integer)
kamus
    i, j : integer
    temp : Buku
algoritma
    for i <- 2 to n do
        temp <- pustaka[i]
        j <- i - 1

        while (j >= 1) and (pustaka[j].rating <
temp.rating) do
            pustaka[j+1] <- pustaka[j]
            j <- j - 1
        endwhile
        pustaka[j+1] <- temp
    endfor
endprocedure

```



```

        endwhile

        pustaka[j+1] <- temp
    endfor
endprocedure

procedure CetakTerfavorit(in pustaka : DaftarBuku, n :
integer)
kamus
    i : integer
algoritma
    for i <- 1 to n do
        output(pustaka[i].judul,
                pustaka[i].penulis,
                pustaka[i].penerbit,
                pustaka[i].tahun,
                pustaka[i].rating)
    endfor
endprocedure

procedure CariBuku(in pustaka : DaftarBuku, n : integer,
rating : integer)
kamus

```

```
kiri, kanan, tengah : integer

ketemu : boolean

algoritma

    kiri <- 1

    kanan <- n

    ketemu <- false

    while (kiri <= kanan) and not ketemu do

        tengah <- (kiri + kanan) div 2

        if pustaka[tengah].rating = rating then

            ketemu <- true

        else if rating > pustaka[tengah].rating then

            kanan <- tengah - 1

        else

            kiri <- tengah + 1

        endif

    endwhile

    if ketemu then

        output(pustaka[tengah].judul,

                pustaka[tengah].penulis,

                pustaka[tengah].penerbit,

                pustaka[tengah].tahun,
```

```
                pustaka[tengah].rating)

    else

        output("Data tidak ditemukan")

    endif
endprocedure
```